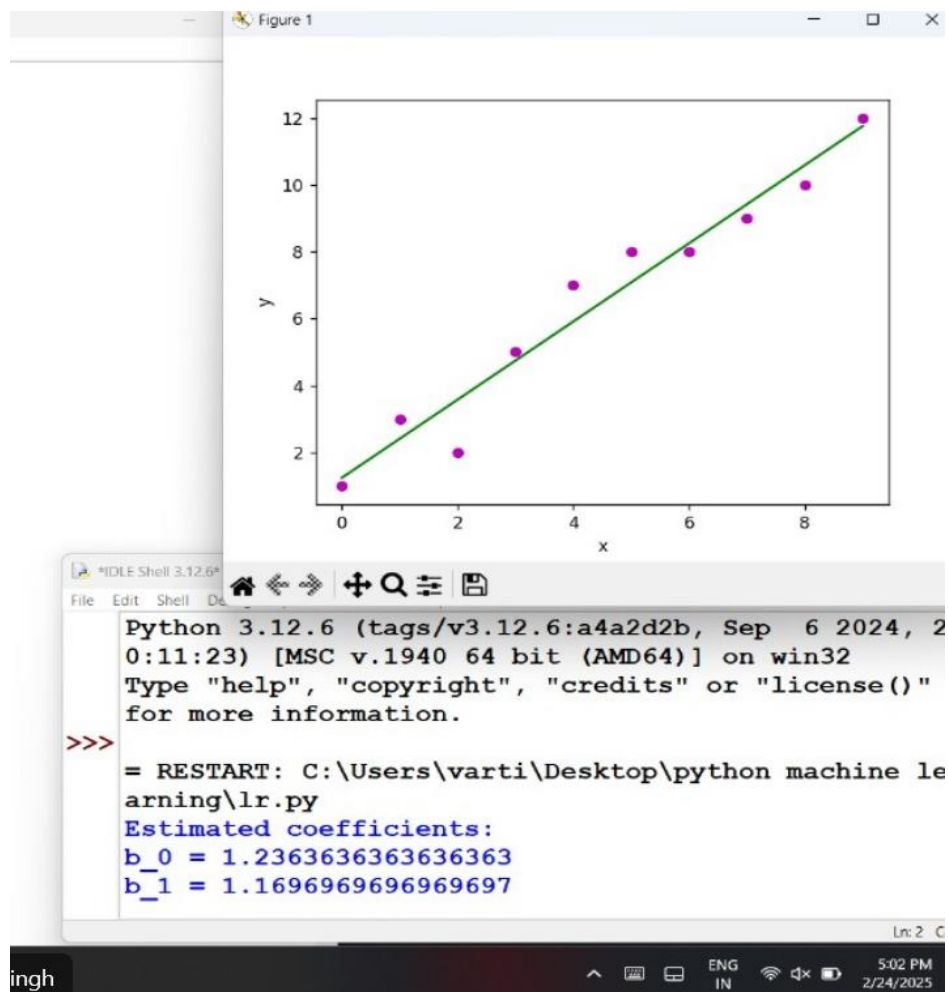


Linear Regression

Code:

```
import numpy as np
import matplotlib.pyplot as plt
def estimate_coef(x, y):
    b_1 = np.cov(x, y, bias=True)[0, 1] / np.var(x)
    return np.mean(y) - b_1 * np.mean(x), b_1
def plot_regression_line(x, y, b):
    plt.scatter(x, y, color="m", marker="o", s=30)
    plt.plot(x, b[0] + b[1]*x, color="g")
    plt.xlabel('x'), plt.ylabel('y'), plt.show()
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])
b = estimate_coef(x, y)
print(f"Estimated coefficients:\nb_0 = {b[0]}\nb_1 = {b[1]}")
plot_regression_line(x, y, b)
```

Output:

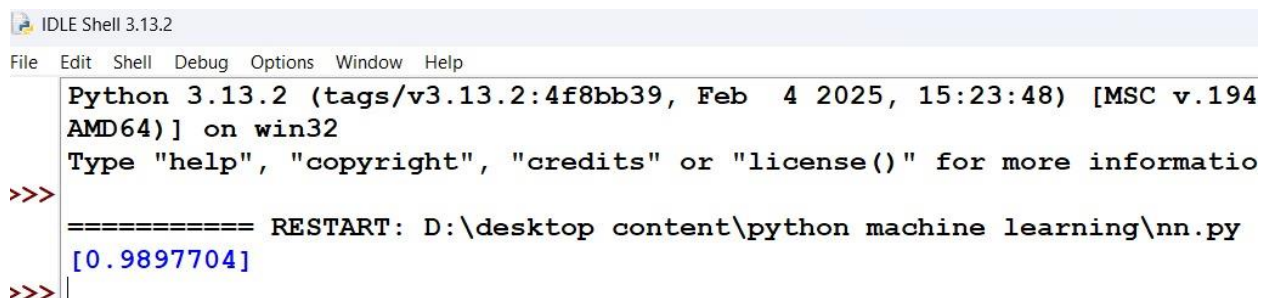


Single-Layer Neural Network

Code:

```
from joblib.numpy_pickle_utils import xrange
from numpy import *
class NeuralNet(object):
    def __init__(self):
        # Generate random numbers
        random.seed(1)
# Assign random weights to a 3 x 1 matrix,
        self.synaptic_weights = 2 * random.random((3, 1)) - 1
# The Sigmoid function
        def __sigmoid(self, x):
            return 1 / (1 + exp(-x))
# The derivative of the Sigmoid function.
        # This is the gradient of the Sigmoid curve.
        def __sigmoid_derivative(self, x):
            return x * (1 - x)
# Train the neural network and adjust the weights each time.
        def train(self, inputs, outputs, training_iterations):
            for iteration in xrange(training_iterations):
                # Pass the training set through the network.
                output = self.learn(inputs)
# Calculate the error
            error = outputs - output
# Adjust the weights by a factor
                factor = dot(inputs.T, error * self.__sigmoid_derivative(output))
                self.synaptic_weights += factor
# The neural network thinks.
        def learn(self, inputs):
            return self.__sigmoid(dot(inputs, self.synaptic_weights))
if __name__ == "__main__":
    # Initialize
    neural_network = NeuralNet()
# The training set.
    inputs = array([[0, 1, 1], [1, 0, 0], [1, 0, 1]])
    outputs = array([[1, 0, 1]]).T
# Train the neural network
    neural_network.train(inputs, outputs, 10000)
# Test the neural network with a test example.
    print(neural_network.learn(array([1, 0, 1])))
```

Output:



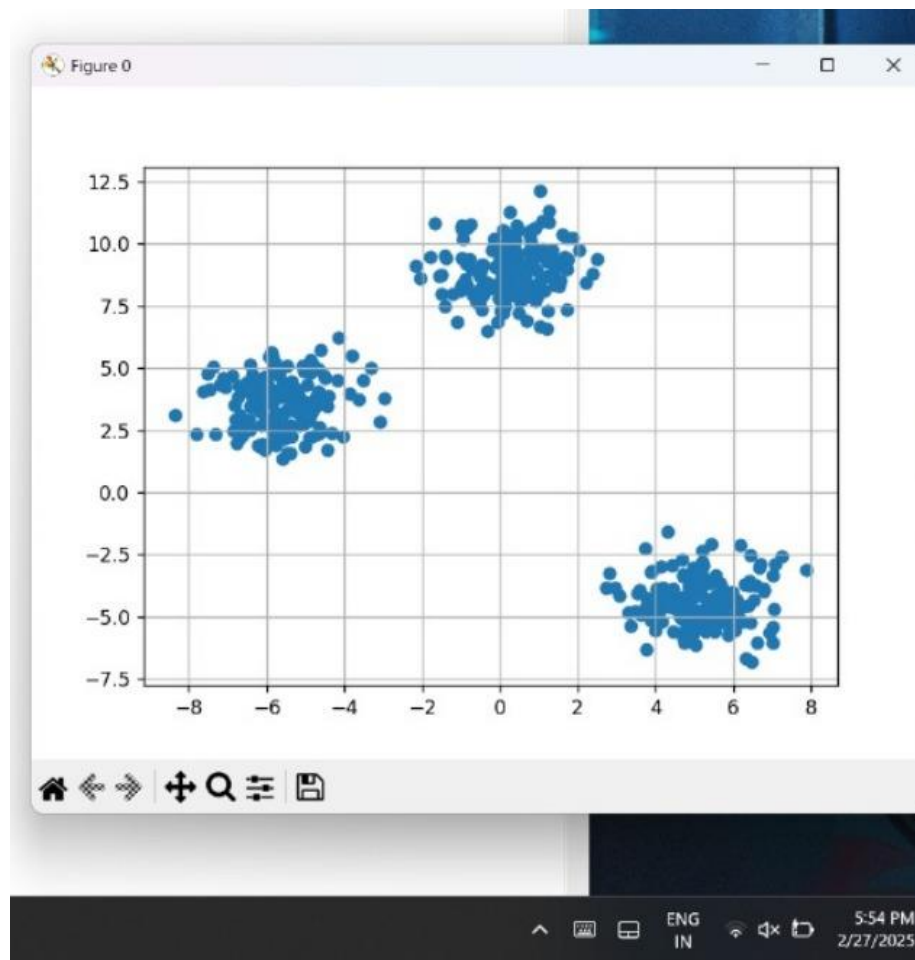
```
IDLE Shell 3.13.2
File Edit Shell Debug Options Window Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb 4 2025, 15:23:48) [MSC v.194
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more informatio
>>>
==== RESTART: D:\desktop content\python machine learning\nn.py
[0.9897704]
>>>|
```

Clustering Algorithms

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
X,y = make_blobs(n_samples = 500,n_features = 2,centers = 3,random_state = 23)
fig = plt.figure(0)
plt.grid(True)
plt.scatter(X[:,0],X[:,1])
plt.show()
```

Output:



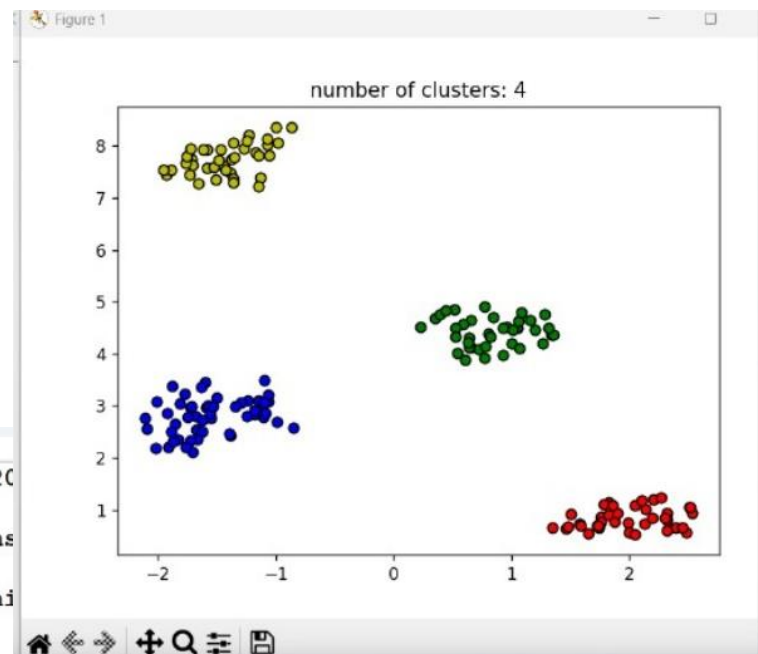
Clustering using DBSCAN

Code:

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn import metrics
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn import datasets
# Load data in X
X, y_true = make_blobs(n_samples=300, centers=4,
                       cluster_std=0.50, random_state=0)
db = DBSCAN(eps=0.3, min_samples=10).fit(X)
core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
core_samples_mask[db.core_sample_indices_] = True
labels = db.labels_
# Number of clusters in labels, ignoring noise if present.
n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
# Plot result
# Black removed and is used for noise instead.
unique_labels = set(labels)
colors = ['y', 'b', 'g', 'r']
print(colors)
for k, col in zip(unique_labels, colors):
    if k == -1:
        # Black used for noise.
        col = 'k'
    class_member_mask = (labels == k)
    xy = X[class_member_mask & core_samples_mask]
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=col,
             markeredgecolor='k',
             markersize=6)
    xy = X[class_member_mask & ~core_samples_mask]
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=col,
             markeredgecolor='k',
             markersize=6)
plt.title('number of clusters: %d' % n_clusters_)
plt.show()
```

Output:

```
File Edit Shell Debug Options Window Help
Python 3.12.6 (tags/v3.12.6:a4a2d2b, Sep 6 2023) on win32
AMD64
Type "help", "copyright", "credits" or "license()"
>>>
= RESTART: C:\Users\varti\Desktop\python machi
['y', 'b', 'g', 'r']
```



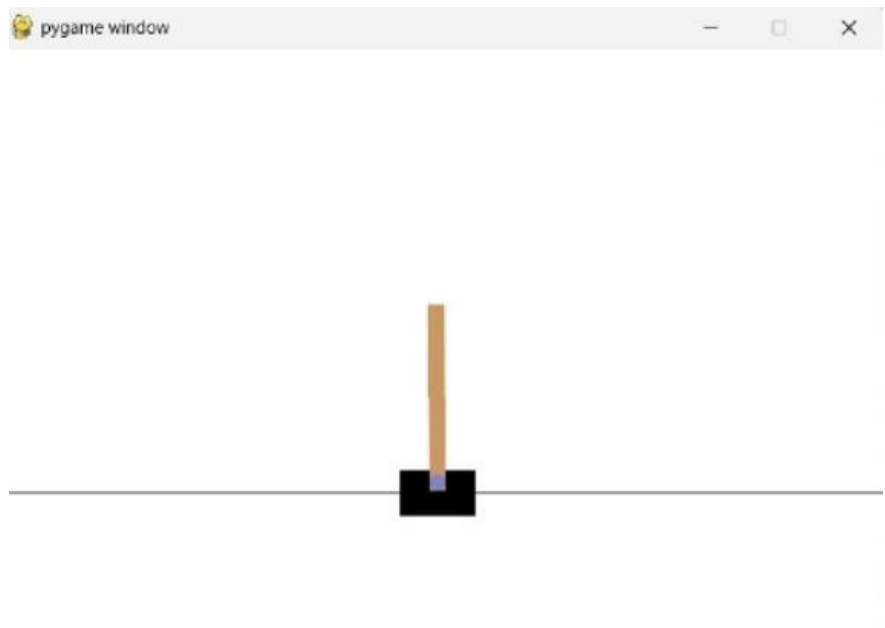
CartPole-v1

Code:

```
import gym
import numpy as np
import warnings
# Suppress specific deprecation warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
# Load the environment with render mode specified
env = gym.make('CartPole-v1', render_mode="human")
# Initialize the environment to get the initial state
state = env.reset()
# Print the state space and action space
print("State space:", env.observation_space)
print("Action space:", env.action_space)
# Run a few steps in the environment with random actions
for _ in range(10):
    env.render() # Render the environment for visualization
    action = env.action_space.sample() # Take a random action
    # Take a step in the environment
    step_result = env.step(action)
    # Check the number of values returned and unpack accordingly
    if len(step_result) == 4:
        next_state, reward, done, info = step_result
        terminated = False
    else:
        next_state, reward, done, truncated, info = step_result
        terminated = done or truncated
    print(f"Action: {action}, Reward: {reward}, Next State: {next_state}, Done: {done}, Info: {info}")

    if terminated:
        state = env.reset() # Reset the environment if the episode is finished
env.close() # Close the environment when done
```

Output:



Evaluating Decision Trees

Code:

```
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
# Function to import the dataset
def importdata():
    balance_data = pd.read_csv(
        'https://archive.ics.uci.edu/ml/machine-learning-databases/balance-scale/balance-scale.data',
        sep=',', header=None)
    print("Dataset Length: ", len(balance_data))
    print("Dataset Shape: ", balance_data.shape)
    print("Dataset: \n", balance_data.head())
    return balance_data
# Function to split the dataset
def splitdataset(balance_data):
    X = balance_data.values[:, 1:5]
    Y = balance_data.values[:, 0]
    X_train, X_test, y_train, y_test = train_test_split(
        X, Y, test_size=0.3, random_state=100)
    return X, Y, X_train, X_test, y_train, y_test
def train_using_gini(X_train, X_test, y_train):
    clf_gini = DecisionTreeClassifier(criterion="gini",
                                     random_state=100, max_depth=3, min_samples_leaf=5)
    clf_gini.fit(X_train, y_train)
    return clf_gini

def train_using_entropy(X_train, X_test, y_train):
    clf_entropy = DecisionTreeClassifier(criterion="entropy",
                                         random_state=100, max_depth=3, min_samples_leaf=5)
    clf_entropy.fit(X_train, y_train)
    return clf_entropy

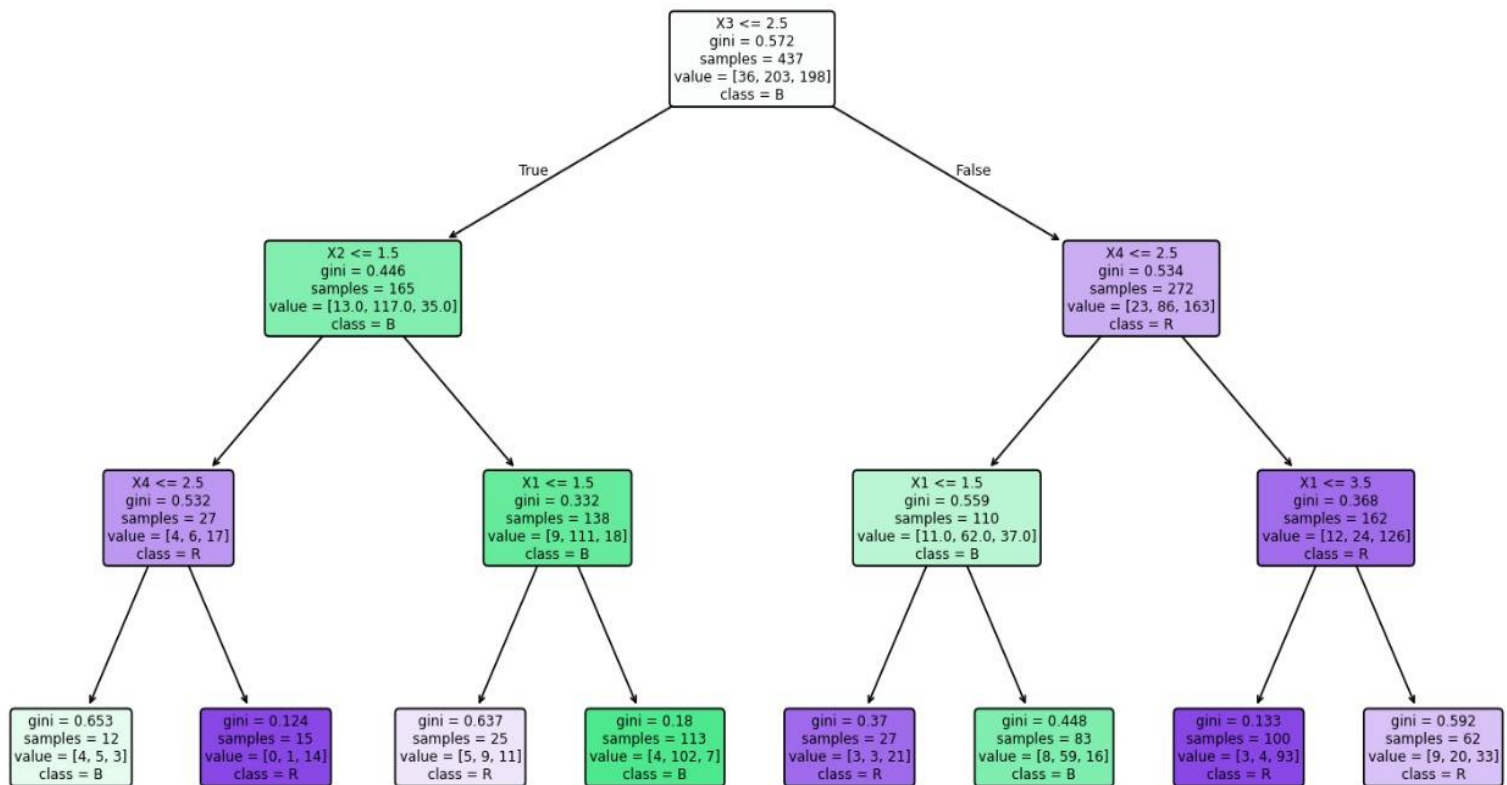
def prediction(X_test, clf_object):
    y_pred = clf_object.predict(X_test)
    print("Predicted values:")
    print(y_pred)
    return y_pred
def cal_accuracy(y_test, y_pred):
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Accuracy : ", accuracy_score(y_test, y_pred)*100)
    print("Report : \n", classification_report(y_test, y_pred))
```

```

def plot_decision_tree(clf_object, feature_names, class_names):
    plt.figure(figsize=(15, 10))
    plot_tree(clf_object, filled=True, feature_names=feature_names, class_names=class_names,
rounded=True)
    plt.show()
# Driver code
if __name__ == "__main__":
    data = importdata()
    X, Y, X_train, X_test, y_train, y_test = splitdataset(data)
    clf_gini = train_using_gini(X_train, X_test, y_train)
    clf_entropy = train_using_entropy(X_train, X_test, y_train)
    print("\nResults Using Gini Index:")
    y_pred_gini = prediction(X_test, clf_gini)
    cal_accuracy(y_test, y_pred_gini)
    print("\nResults Using Entropy:")
    y_pred_entropy = prediction(X_test, clf_entropy)
    cal_accuracy(y_test, y_pred_entropy)
    plot_decision_tree(clf_gini, ['X1', 'X2', 'X3', 'X4'], ['L', 'B', 'R'])
    plot_decision_tree(clf_entropy, ['X1', 'X2', 'X3', 'X4'], ['L', 'B', 'R'])

```

Output:

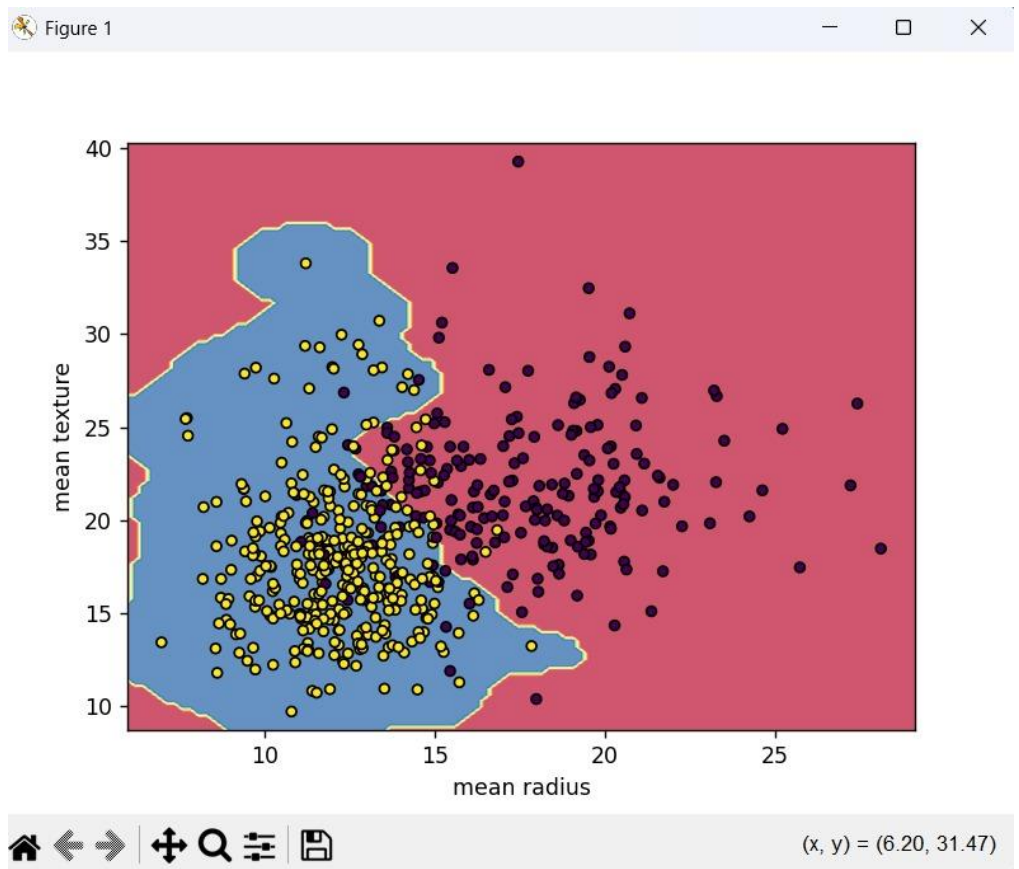


SVM with RBF kernel

Code:

```
from sklearn.datasets import load_breast_cancer
import matplotlib.pyplot as plt
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.svm import SVC
# Load the datasets
cancer = load_breast_cancer()
X = cancer.data[:, :2]
y = cancer.target
#Build the model
svm = SVC(kernel="rbf", gamma=0.5, C=1.0)
# Trained the model
svm.fit(X, y)
# Plot Decision Boundary
DecisionBoundaryDisplay.from_estimator(
    svm,
    X,
    response_method="predict",
    cmap=plt.cm.Spectral,
    alpha=0.8,
    xlabel=cancer.feature_names[0],
    ylabel=cancer.feature_names[1],
)
# Scatter plot
plt.scatter(X[:, 0], X[:, 1],
            c=y,
            s=20, edgecolors="k")
plt.show()
```

Output:



Genetic Algorithm for Solving the Travelling Salesman Problem (TSP)

Code:

```
# Python3 implementation
from random import randint
INT_MAX = 2147483647
# Number of cities in TSP
V = 5
# Names of the cities
GENES = "ABCDE"
# Starting Node Value
START = 0
# Initial population size for the algorithm
POP_SIZE = 10
# Structure of a GNOME
# defines the path traversed
# by the salesman while the fitness value
# of the path is stored in an integer
class individual:
    def __init__(self) -> None:
        self.gnome = ""
        self.fitness = 0
    def __lt__(self, other):
        return self.fitness < other.fitness
    def __gt__(self, other):
        return self.fitness > other.fitness
# Function to return a random number
# from start and end
def rand_num(start, end):
    return randint(start, end-1)
# Function to check if the character
# has already occurred in the string
def repeat(s, ch):
    for i in range(len(s)):
        if s[i] == ch:
            return True
    return False
# Function to return a mutated GNOME
# Mutated GNOME is a string
# with a random interchange
# of two genes to create variation in species
def mutatedGene(gnome):
    gnome = list(gnome)
    while True:
        r = rand_num(1, V)
        r1 = rand_num(1, V)
        if r1 != r:
```

```

        temp = gnome[r]
        gnome[r] = gnome[r1]
        gnome[r1] = temp
        break
    return "".join(gnome)
# Function to return a valid GNOME string
# required to create the population
def create_gnome():
    gnome = "0"
    while True:
        if len(gnome) == V:
            gnome += gnome[0]
            break
        temp = rand_num(1, V)
        if not repeat(gnome, chr(temp + 48)):
            gnome += chr(temp + 48)
    return gnome
# Function to return the fitness value of a gnome.
# The fitness value is the path length
# of the path represented by the GNOME.
def cal_fitness(gnome):
    mp = [
        [0, 2, INT_MAX, 12, 5],
        [2, 0, 4, 8, INT_MAX],
        [INT_MAX, 4, 0, 3, 3],
        [12, 8, 3, 0, 10],
        [5, INT_MAX, 3, 10, 0],
    ]
    f = 0
    for i in range(len(gnome) - 1):
        if mp[ord(gnome[i]) - 48][ord(gnome[i + 1]) - 48] == INT_MAX:
            return INT_MAX
        f += mp[ord(gnome[i]) - 48][ord(gnome[i + 1]) - 48]
    return f
# Function to return the updated value
# of the cooling element.
def cooldown(temp):
    return (90 * temp) / 100
# Comparator for GNOME struct.
# def lessthan(individual t1,
#             individual t2)
# :
# return t1.fitness < t2.fitness
# Utility function for TSP problem.
def TSPUtil(mp):
    # Generation Number

```

```

gen = 1
# Number of Gene Iterations
gen_thres = 5
population = []
temp = individual()
# Populating the GNOME pool.
for i in range(POP_SIZE):
    temp.gnome = create_gnome()
    temp.fitness = cal_fitness(temp.gnome)
    population.append(temp)
print("\nInitial population: \nGNOME    FITNESS VALUE\n")
for i in range(POP_SIZE):
    print(population[i].gnome, population[i].fitness)
print()
found = False
temperature = 10000
# Iteration to perform
# population crossing and gene mutation.
while temperature > 1000 and gen <= gen_thres:
    population.sort()
    print("\nCurrent temp: ", temperature)
    new_population = []
    for i in range(POP_SIZE):
        p1 = population[i]
        while True:
            new_g = mutatedGene(p1.gnome)
            new_gnome = individual()
            new_gnome.gnome = new_g
            new_gnome.fitness = cal_fitness(new_gnome.gnome)
            if new_gnome.fitness <= population[i].fitness:
                new_population.append(new_gnome)
                break
        else:
            # Accepting the rejected children at
            # a possible probability above threshold.
            prob = pow(
                2.7,
                -1
                * (
                    (float)(new_gnome.fitness - population[i].fitness)
                    / temperature
                ),
            )
            if prob > 0.5:
                new_population.append(new_gnome)
                break

```

```

    temperature = cooldown(temperature)
    population = new_population
    print("Generation", gen)
    print("GNOME    FITNESS VALUE")
    for i in range(POP_SIZE):
        print(population[i].gnome, population[i].fitness)
    gen += 1
if __name__ == "__main__":
    mp = [
        [0, 2, INT_MAX, 12, 5],
        [2, 0, 4, 8, INT_MAX],
        [INT_MAX, 4, 0, 3, 3],
        [12, 8, 3, 0, 10],
        [5, INT_MAX, 3, 10, 0],
    ]
    TSPUtil(mp)

```

Output:

```

12:55 Tue, 11 Mar
←

Initial population:
GNOME    FITNESS VALUE

024310 2147483647
024310 2147483647
024310 2147483647
024310 2147483647
024310 2147483647
024310 2147483647
024310 2147483647
024310 2147483647
024310 2147483647

Current temp: 10000
Generation 1
GNOME    FITNESS VALUE

```

```

12:55 Tue, 11 Mar
←

Current temp: 10000
Generation 1
GNOME    FITNESS VALUE
024130 2147483647
024130 2147483647
024130 2147483647
021340 2147483647
014320 2147483647
023410 2147483647
014320 2147483647
021340 2147483647
042310 21
042310 21

Current temp: 9000.0
Generation 2
GNOME    FITNESS VALUE
043210 24
042130 32
023140 2147483647
034120 2147483647
023140 2147483647
031240 32
024310 2147483647
023140 2147483647
041320 2147483647
023140 2147483647

Current temp: 8100.0
Generation 3
GNOME    FITNESS VALUE
034210 31
012430 31
034210 31
021340 2147483647
034210 31
032140 2147483647
024130 2147483647
023410 2147483647
041230 2147483647
032140 2147483647

Current temp: 7290.0
Generation 4
GNOME    FITNESS VALUE
031240 32
012340 24
043210 24
043210 24
031240 32
034120 2147483647
021430 2147483647
023140 2147483647
043210 24
012340 24

Current temp: 6561.0
Generation 5
GNOME    FITNESS VALUE
042310 21
013240 21
013240 21
042310 21
012430 31
013240 21
034210 31
034210 31
041230 2147483647
043120 2147483647

[Program finished]

```